1.7. Gaussian Processes

Gaussian Processes (GP) are a nonparametric supervised learning method used to solve *regression* and *probabilistic classification* problems.

The advantages of Gaussian processes are:

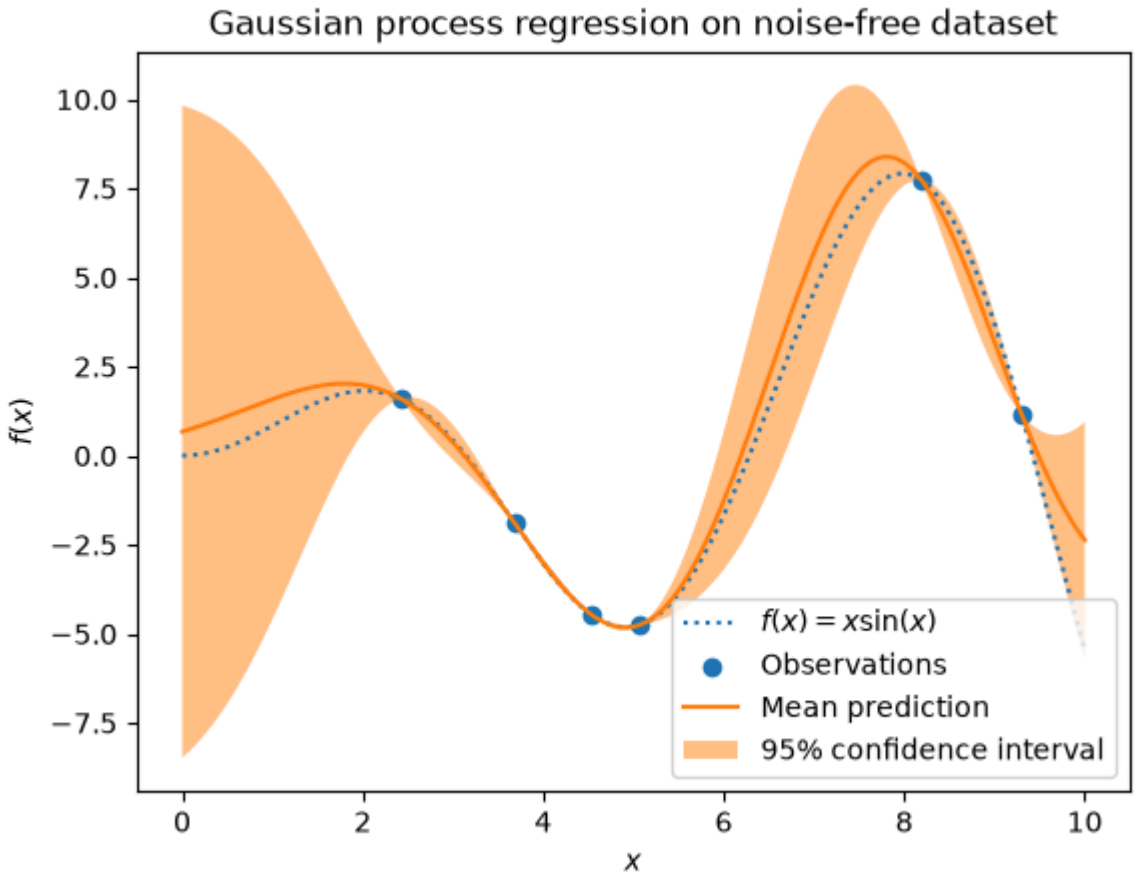
- The prediction interpolates the observations (at least for regular kernels).
- The prediction is probabilistic (Gaussian) so that one can compute empirical confidence intervals and decide based on those if one should refit (online fitting, adaptive fitting) the prediction in some region of interest.
- Versatile: different kernels can be specified. Common kernels are provided, but it is also possible to specify custom kernels.

The disadvantages of Gaussian processes include:

- Our implementation is not sparse, i.e., they use the whole samples/features information to perform the prediction.
- They lose efficiency in high dimensional spaces – namely when the number of features exceeds a few dozens.

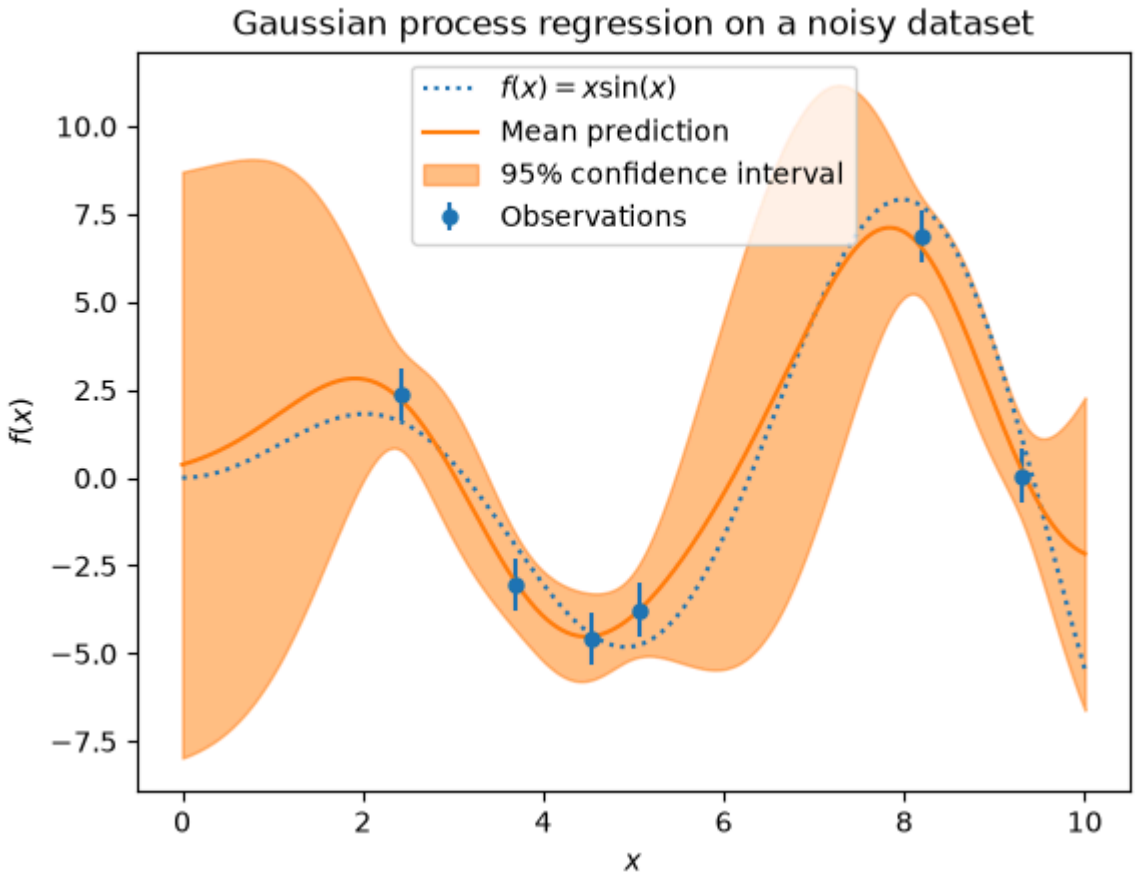
1.7.1. Gaussian Process Regression (GPR)

The `GaussianProcessRegressor` implements Gaussian processes (GP) for regression purposes. For this, the prior of the GP needs to be specified. GP will combine this prior and the likelihood function based on training samples. It allows to give a probabilistic approach to prediction by giving the mean and standard deviation as output when predicting.



The prior mean is assumed to be constant and zero (for `normalize_y=False`) or the training data's mean (for `normalize_y=True`). The prior's covariance is specified by passing a kernel object. The hyperparameters of the kernel are optimized when fitting the **`GaussianProcessRegressor`** by maximizing the log-marginal-likelihood (LML) based on the passed `optimizer`. As the LML may have multiple local optima, the optimizer can be started repeatedly by specifying `n_restarts_optimizer`. The first run is always conducted starting from the initial hyperparameter values of the kernel; subsequent runs are conducted from hyperparameter values that have been chosen randomly from the range of allowed values. If the initial hyperparameters should be kept fixed, `None` can be passed as optimizer.

The noise level in the targets can be specified by passing it via the parameter `alpha`, either globally as a scalar or per datapoint. Note that a moderate noise level can also be helpful for dealing with numeric instabilities during fitting as it is effectively implemented as Tikhonov regularization, i.e., by adding it to the diagonal of the kernel matrix. An alternative to specifying the noise level explicitly is to include a **`WhiteKernel`** component into the kernel, which can estimate the global noise level from the data (see example below). The figure below shows the effect of noisy target handled by setting the parameter `alpha`.



The implementation is based on Algorithm 2.1 of [RW2006]. In addition to the API of standard scikit-learn estimators, `GaussianProcessRegressor`:

- allows prediction without prior fitting (based on the GP prior)
- provides an additional method `sample_y(X)`, which evaluates samples drawn from the GPR (prior or posterior) at given inputs
- exposes a method `log_marginal_likelihood(theta)`, which can be used externally for other ways of selecting hyperparameters, e.g., via Markov chain Monte Carlo.

Examples

- Gaussian Processes regression: basic introductory example
- Ability of Gaussian process regression (GPR) to estimate data noise-level
- Comparison of kernel ridge and Gaussian process regression
- Forecasting of CO2 level on Mona Loa dataset using Gaussian process regression (GPR)

1.7.2. Gaussian Process Classification (GPC)

The **GaussianProcessClassifier** implements Gaussian processes (GP) for classification purposes, more specifically for probabilistic classification, where test predictions take the form of class probabilities. GaussianProcessClassifier places a GP prior on a latent function f, which is then squashed through a link function π to obtain the probabilistic classification. The latent function f is a so-called nuisance function, whose values are not observed and are not relevant by themselves. Its purpose is to allow a convenient formulation of the model, and f is removed (integrated out) during prediction. **GaussianProcessClassifier** implements the logistic link function, for which the integral cannot be computed analytically but is easily approximated in the binary case.

In contrast to the regression setting, the posterior of the latent function f is not Gaussian even for a GP prior since a Gaussian likelihood is inappropriate for discrete class labels. Rather, a non-Gaussian likelihood corresponding to the logistic link function (logit) is used. GaussianProcessClassifier approximates the non-Gaussian posterior with a Gaussian based on the Laplace approximation. More details can be found in Chapter 3 of [RW2006].

The GP prior mean is assumed to be zero. The prior's covariance is specified by passing a kernel object. The hyperparameters of the kernel are optimized during fitting of GaussianProcessRegressor by maximizing the log-marginal-likelihood (LML) based on the passed `optimizer`. As the LML may have multiple local optima, the optimizer can be started repeatedly by specifying `n_restarts_optimizer`. The first run is always conducted starting from the initial hyperparameter values of the kernel; subsequent runs are conducted from hyperparameter values that have been chosen randomly from the range of allowed values. If the initial hyperparameters should be kept fixed, `None` can be passed as optimizer.

In some scenarios, information about the latent function f is desired (i.e. the mean $\bar{f}_*$ and the variance $\mathrm{Var}[f_*]$ described in Eqs. (3.21) and (3.24) of [RW2006]). The **GaussianProcessClassifier** provides access to these quantities via the `latent_mean_and_variance` method.

GaussianProcessClassifier supports multi-class classification by performing either one-versus-rest or one-versus-one based training and prediction. In one-versus-rest, one binary Gaussian process classifier is fitted for each class, which is trained to separate this class from the rest. In "one_vs_one", one binary Gaussian process classifier is fitted for each pair of classes, which is trained to separate these two classes. The predictions of these binary predictors are combined into multi-class predictions. See the section on multi-class classification for more details.

In the case of Gaussian process classification, "one_vs_one" might be computationally cheaper since it has to solve many problems involving only a subset of the whole training set rather than fewer problems on the whole dataset. Since Gaussian process classification scales cubically with

the size of the dataset, this might be considerably faster. However, note that “one_vs_one” does not support predicting probability estimates but only plain predictions. Moreover, note that `GaussianProcessClassifier` does not (yet) implement a true multi-class Laplace approximation internally, but as discussed above is based on solving several binary classification tasks internally, which are combined using one-versus-rest or one-versus-one.

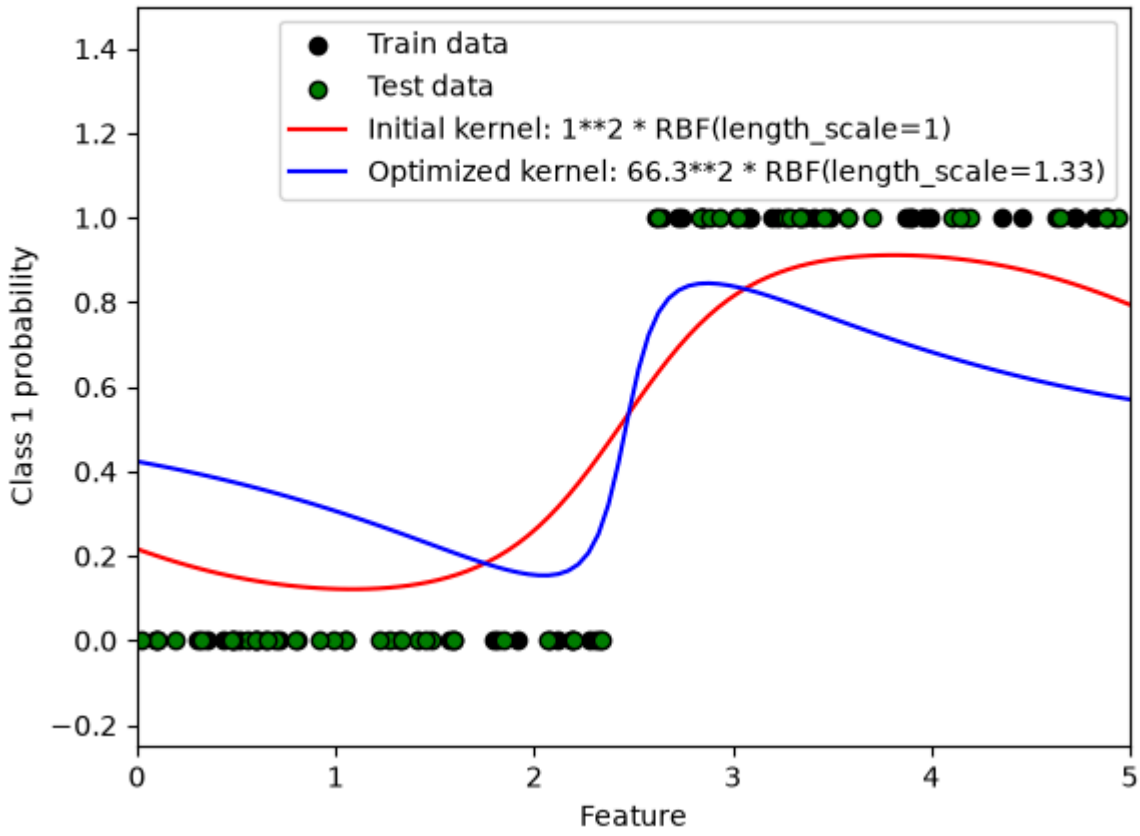
1.7.3. GPC examples

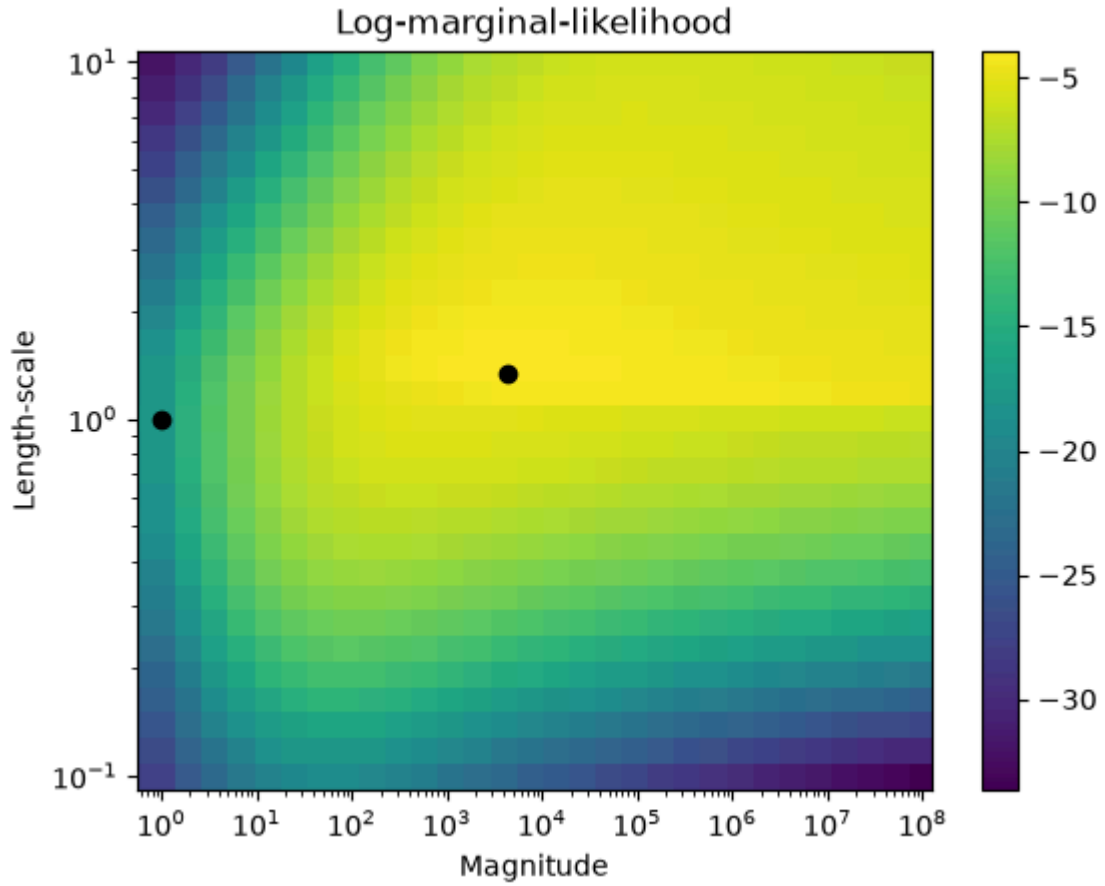
1.7.3.1. Probabilistic predictions with GPC

This example illustrates the predicted probability of GPC for an RBF kernel with different choices of the hyperparameters. The first figure shows the predicted probability of GPC with arbitrarily chosen hyperparameters and with the hyperparameters corresponding to the maximum log-marginal-likelihood (LML).

While the hyperparameters chosen by optimizing LML have a considerably larger LML, they perform slightly worse according to the log-loss on test data. The figure shows that this is because they exhibit a steep change of the class probabilities at the class boundaries (which is good) but have predicted probabilities close to 0.5 far away from the class boundaries (which is bad). This undesirable effect is caused by the Laplace approximation used internally by GPC.

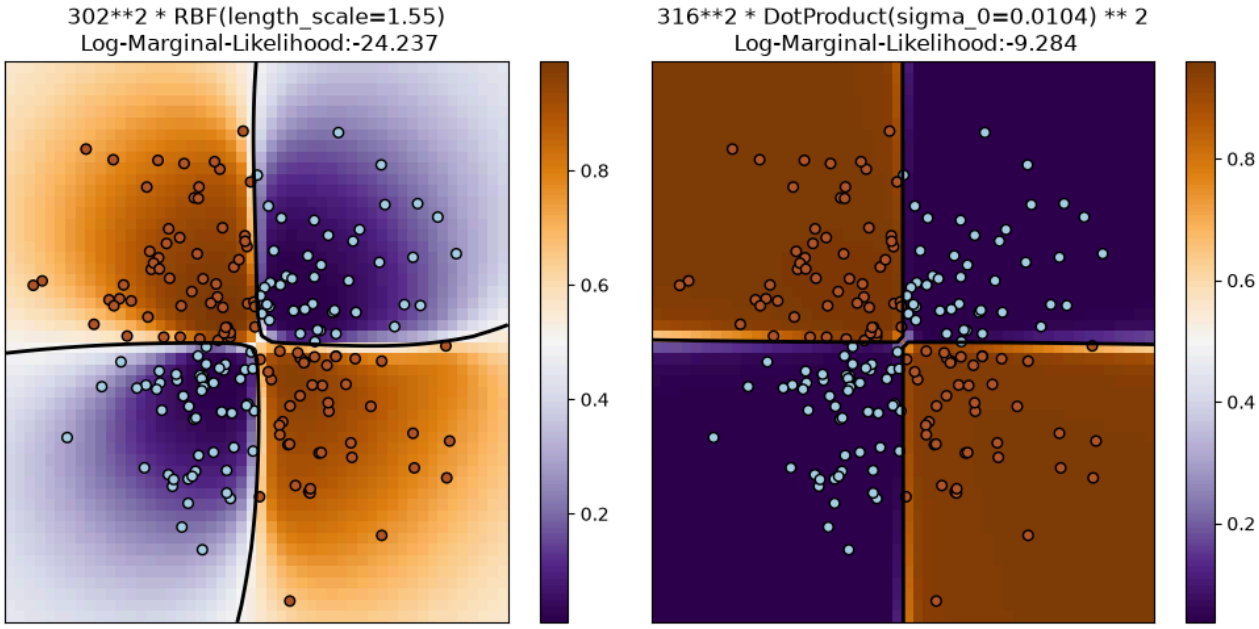
The second figure shows the log-marginal-likelihood for different choices of the kernel’s hyperparameters, highlighting the two choices of the hyperparameters used in the first figure by black dots.


Train data
Test data
Initial kernel: 1**2 * RBF(length_scale=1)
Optimized kernel: 66.3**2 * RBF(length_scale=1.33)
Class 1 probability
Feature
1.4
1.2
1.0
0.8
0.6
0.4
0.2
0.0
−0.2
0
1
2
3
4
5


Log-marginal-likelihood
Length-scale
Magnitude
10^{1}
10^{0}
10^{-1}
10^{0} 10^{1} 10^{2} 10^{3} 10^{4} 10^{5} 10^{6} 10^{7} 10^{8}
−5
−10
−15
−20
−25
−30

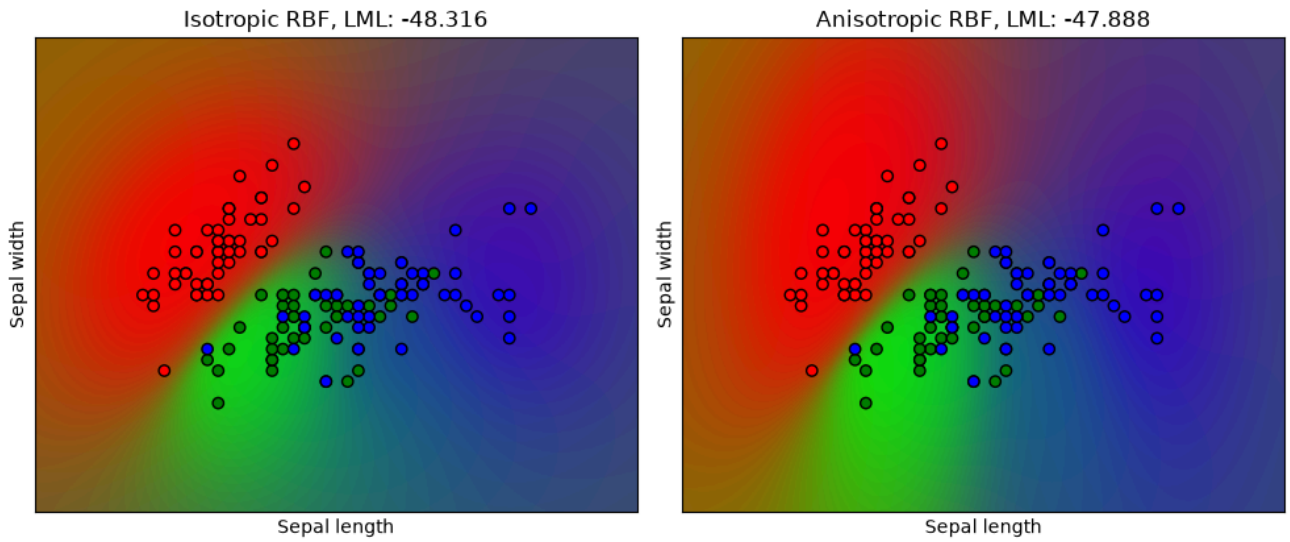
1.7.3.2. Illustration of GPC on the XOR dataset

This example illustrates GPC on XOR data. Compared are a stationary, isotropic kernel (`RBF`) and a non-stationary kernel (`DotProduct`). On this particular dataset, the `DotProduct` kernel obtains considerably better results because the class-boundaries are linear and coincide with the coordinate axes. In practice, however, stationary kernels such as `RBF` often obtain better results.



1.7.3.3. Gaussian process classification (GPC) on iris dataset

This example illustrates the predicted probability of GPC for an isotropic and anisotropic RBF kernel on a two-dimensional version for the iris dataset. This illustrates the applicability of GPC to non-binary classification. The anisotropic RBF kernel obtains slightly higher log-marginal-likelihood by assigning different length-scales to the two feature dimensions.



1.7.4. Kernels for Gaussian Processes

Kernels (also called "covariance functions" in the context of GPs) are a crucial ingredient of GPs which determine the shape of prior and posterior of the GP. They encode the assumptions on the function being learned by defining the "similarity" of two datapoints combined with the assumption that similar datapoints should have similar target values. Two categories of kernels can be distinguished: stationary kernels depend only on the distance of two datapoints and not on their absolute values $k(x_i, x_j) = k(d(x_i, x_j))$ and are thus invariant to translations in the input space, while non-stationary kernels depend also on the specific values of the datapoints. Stationary kernels can further be subdivided into isotropic and anisotropic kernels, where isotropic kernels are also invariant to rotations in the input space. For more details, we refer to Chapter 4 of [RW2006]. This example shows how to define a custom kernel over discrete data. For guidance on how to best combine different kernels, we refer to [Duv2014].

Gaussian Process Kernel API

1.7.4.1. Basic kernels

The `ConstantKernel` kernel can be used as part of a `Product` kernel where it scales the magnitude of the other factor (kernel) or as part of a `Sum` kernel, where it modifies the mean of the Gaussian process. It depends on a parameter $constant_value$. It is defined as:

$$k(x_i, x_j) = constant_value \;\forall\; x_i, x_j$$

The main use-case of the `WhiteKernel` kernel is as part of a sum-kernel where it explains the noise-component of the signal. Tuning its parameter $noise_level$ corresponds to estimating the noise-level. It is defined as:

$$k(x_i, x_j) = noise_level \text{ if } x_i == x_j \text{ else } 0$$

1.7.4.2. Kernel operators

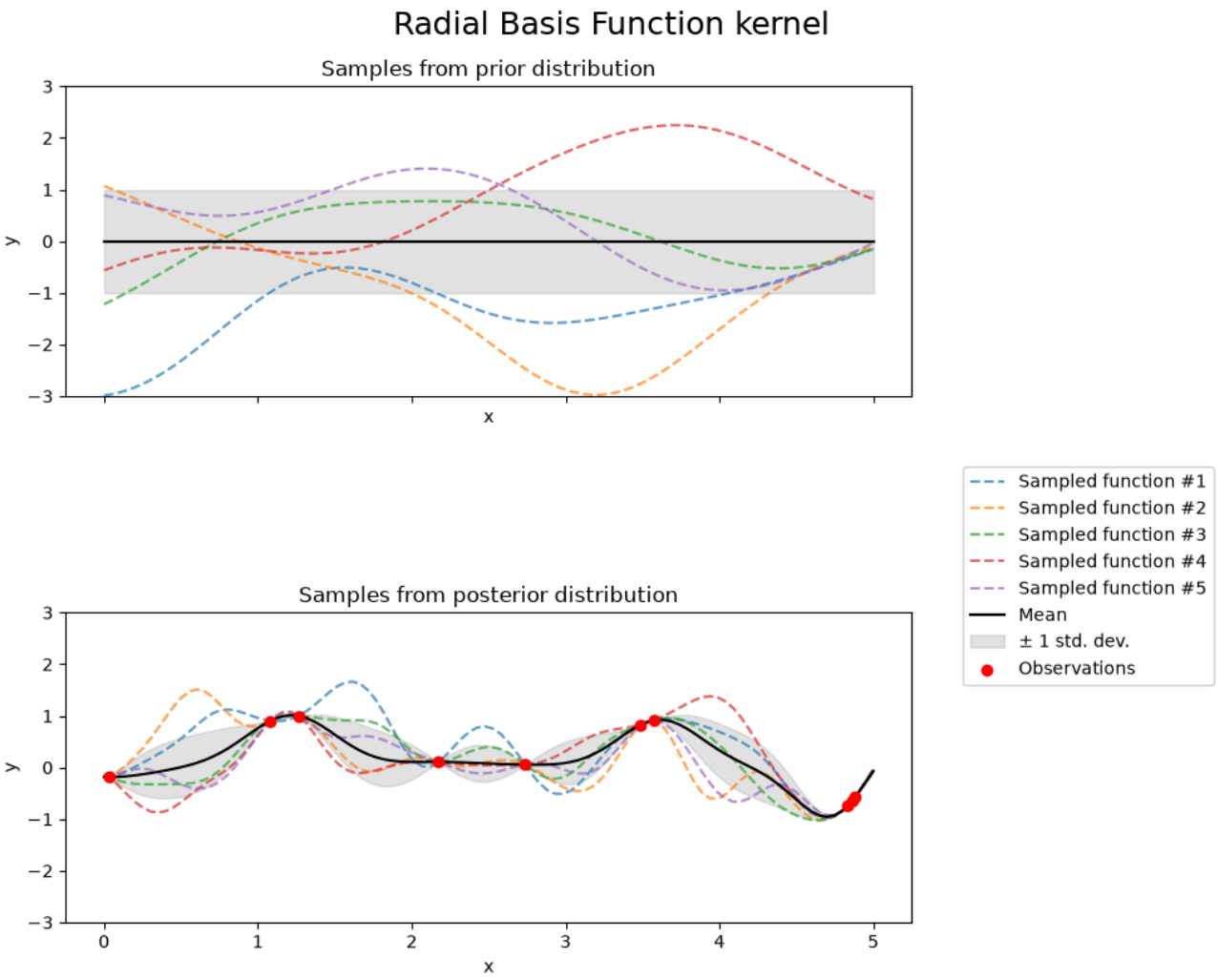
Kernel operators take one or two base kernels and combine them into a new kernel. The `Sum` kernel takes two kernels k_1 and k_2 and combines them via $k_{sum}(X, Y) = k_1(X, Y) + k_2(X, Y)$. The `Product` kernel takes two kernels k_1 and k_2 and combines them via $k_{product}(X, Y) = k_1(X, Y) * k_2(X, Y)$. The `Exponentiation` kernel takes one base kernel and a scalar parameter p and combines them via $k_{exp}(X, Y) = k(X, Y)^p$. Note that magic methods `__add__`, `__mul___` and `__pow__` are overridden on the Kernel objects, so one can use e.g. `RBF() + RBF()` as a shortcut for `Sum(RBF(), RBF())`.

1.7.4.3. Radial basis function (RBF) kernel

The `RBF` kernel is a stationary kernel. It is also known as the "squared exponential" kernel. It is parameterized by a length-scale parameter $l > 0$, which can either be a scalar (isotropic variant of the kernel) or a vector with the same number of dimensions as the inputs x (anisotropic variant of the kernel). The kernel is given by:

$$k(x_i, x_j) = \exp\left(-\frac{d(x_i, x_j)^2}{2l^2}\right)$$

where $d(\cdot, \cdot)$ is the Euclidean distance. This kernel is infinitely differentiable, which implies that GPs with this kernel as covariance function have mean square derivatives of all orders, and are thus very smooth. The prior and posterior of a GP resulting from an RBF kernel are shown in the following figure:

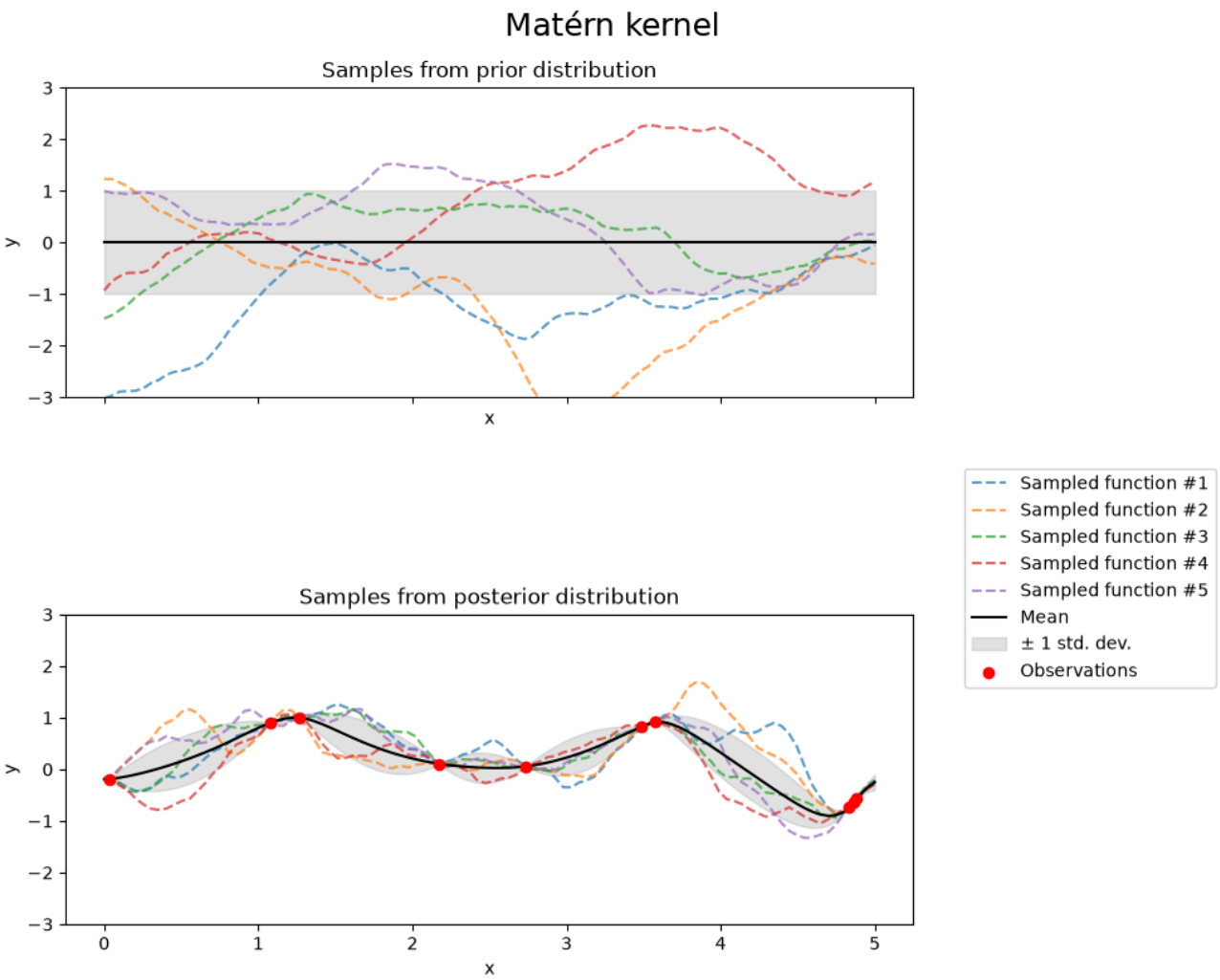


1.7.4.4. Matérn kernel

The `Matern` kernel is a stationary kernel and a generalization of the `RBF` kernel. It has an additional parameter ν which controls the smoothness of the resulting function. It is parameterized by a length-scale parameter $l > 0$, which can either be a scalar (isotropic variant of the kernel) or a vector with the same number of dimensions as the inputs x (anisotropic variant of the kernel).

Mathematical implementation of Matérn kernel

The prior and posterior of a GP resulting from a Matérn kernel are shown in the following figure:



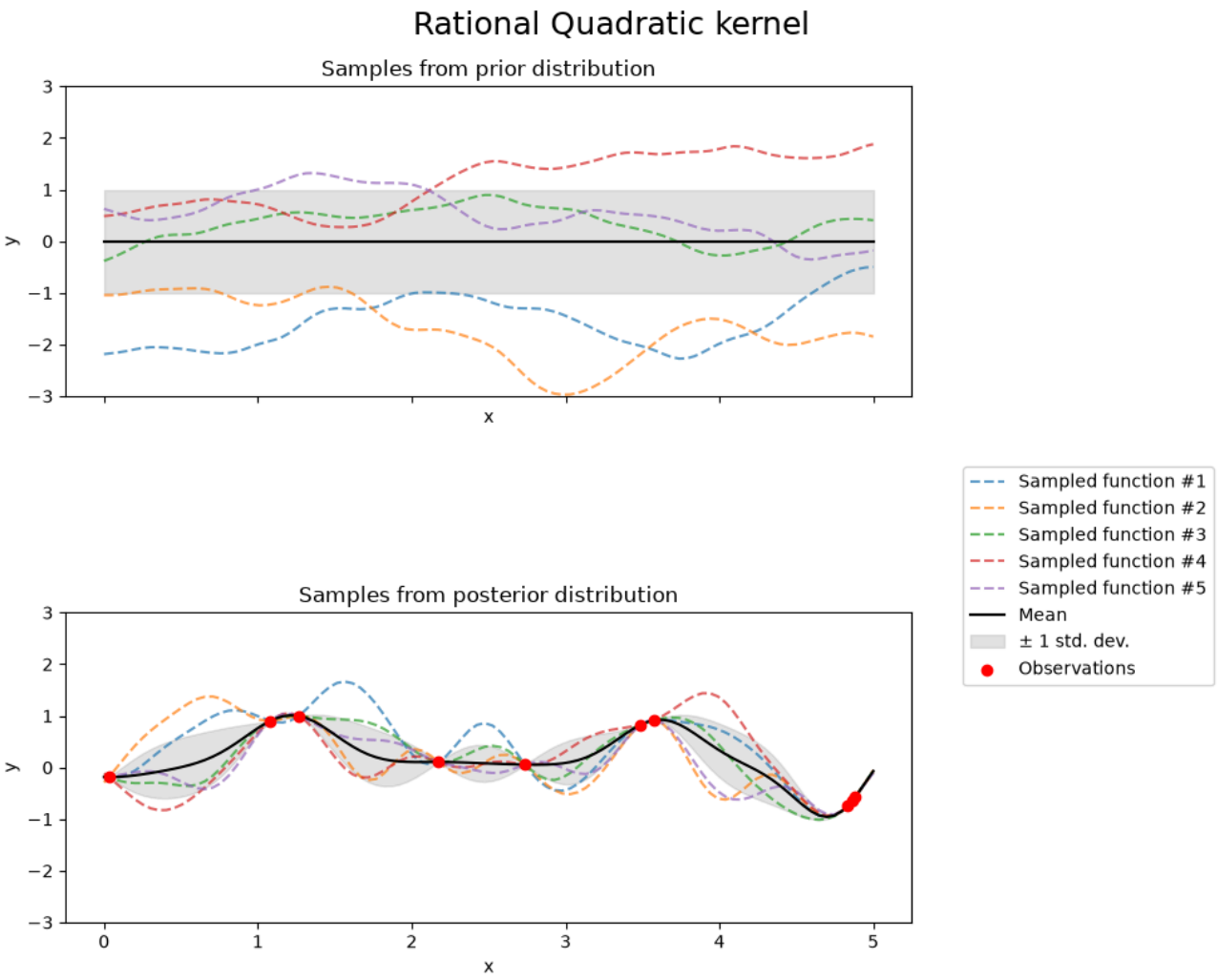
See [RW2006], pp84 for further details regarding the different variants of the Matérn kernel.

1.7.4.5. Rational quadratic kernel

The `RationalQuadratic` kernel can be seen as a scale mixture (an infinite sum) of `RBF` kernels with different characteristic length-scales. It is parameterized by a length-scale parameter $l > 0$ and a scale mixture parameter $\alpha > 0$ Only the isotropic variant where l is a scalar is supported at the moment. The kernel is given by:

$$k(x_i, x_j) = \left(1 + \frac{d(x_i, x_j)^2}{2\alpha l^2}\right)^{-\alpha}$$

The prior and posterior of a GP resulting from a `RationalQuadratic` kernel are shown in the following figure:

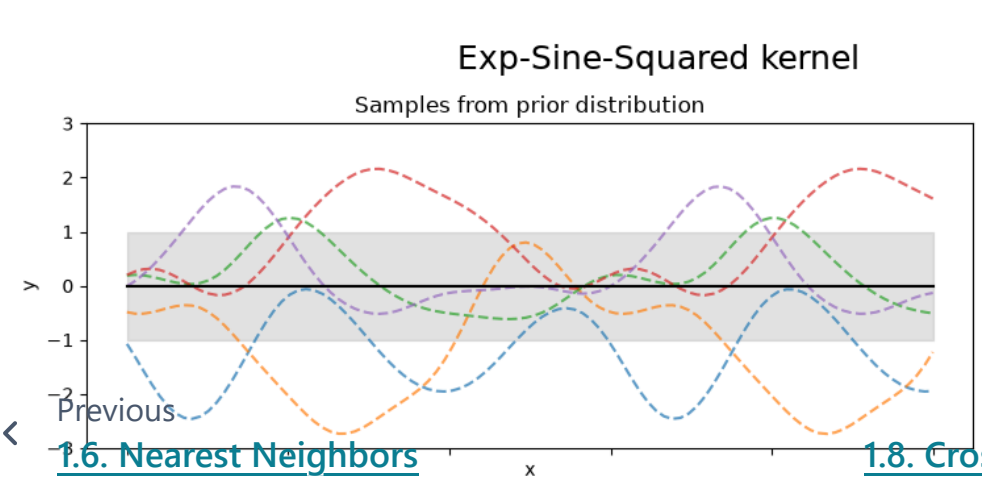


1.7.4.6. Exp-Sine-Squared kernel

The `ExpSineSquared` kernel allows modeling periodic functions. It is parameterized by a length-scale parameter $l > 0$ and a periodicity parameter $p > 0$. Only the isotropic variant where l is a scalar is supported at the moment. The kernel is given by:

$$k(x_i, x_j) = \exp\left(-\frac{2\sin^2(\pi d(x_i, x_j)/p)}{l^2}\right)$$

The prior and posterior of a GP resulting from an ExpSineSquared kernel are shown in the following figure:



Previous
1.6. Nearest Neighbors
Next
1.8. Cross decomposition

© Copyright 2007 - 2026, scikit-learn developers (BSD License).